```java
public class LinkedList
{
    private Node head;     22 usages
    private Node tail;  13 usages
    private int length;  19 usages


    class Node  36 usages
    {
        int value;  9 usages
        Node next;  28 usages

        Node(int value) { this.value = value; }
    }

    // Constructor: crea la lista con un primer nodo que contiene "value"
    public LinkedList(int value)  2 usages
    {
        Node newNode = new Node(value);
        this.length = 1;
        this.head = newNode;
        this.tail = newNode;
    }

    // Regresa el primer nodo (o null si la lista está vacía)
    public Node getHead() { return this.head; }

    // Asigna manualmente el nodo cabeza
    public void setHead(Node head) { this.head = head; }

    // Asigna manualmente el nodo cola
```

```java
    // Asigna manualmente el nodo cola
    public void setTail(Node tail) { this.tail = tail; }

    // Regresa el último nodo (o null si la lista está vacía).
    // Se corrigió: el original no regresaba nada cuando tail era null y no compilaba.
    public Node getTail() { return this.tail; }

    // Regresa la cantidad de nodos de la lista
    public int getLength() { return this.length; }

    // Append(valor): agrega un nodo al final de la lista.
    // Si la lista está vacía, el nuevo nodo es a la vez head y tail.
    // Si no, se enlaza después de tail y tail pasa a ser el nuevo nodo.
    public void append(int value)
    {
        Node newNode = new Node(value);
        if (length == 0)
        {
            this.head = newNode;
            this.tail = newNode;
        }
        else
        {
            this.tail.next = newNode;
            this.tail = newNode;
        }
        length++;
    }
```

```java
    }

    // Prepend(valor): agrega un nodo al inicio de la lista.
    // El nuevo nodo apunta al antiguo head y se convierte en el nuevo head.
    // Si la lista estaba vacía, también es la tail.
    public void prepend(int value)  3 usages
    {
        Node newNode = new Node(value);
        if (length == 0)
        {
            this.head = newNode;
            this.tail = newNode;
        }
        else
        {
            newNode.next = this.head;
            this.head = newNode;
        }
        length++;
    }

    // Insert(indice, valor): agrega un nodo en la posición "index".
    // Regresa false si el índice es inválido (menor que 0 o mayor que length).
    // - index 0      -> equivale a prepend
    // - index length -> equivale a append
    // - otro caso    -> se llega al nodo anterior (index-1) y se enlaza el nuevo nodo en medio
    public boolean insert(int index, int value)  4 usages
    {
        if (index < 0 || index > length) return false;
        if (index == 0)
```

```
106         {
107             prepend(value);
108             return true;
109         }
110         if (index == length)
111         {
112             append(value);
113             return true;
114         }
115         Node newNode = new Node(value);
116         Node pre = getNode( index: index - 1);
117         newNode.next = pre.next;
118         pre.next = newNode;
119         length++;
120         return true;
121     }
122 
123     // RemoveFirst(): elimina y regresa el primer nodo (null si la lista está vacía).
124     // head avanza al siguiente nodo y el nodo removido se desconecta.
125     // Si la lista queda vacía, tail también se pone en null.
126     public Node removeFirst()  3 usages
127     {
128         if (length == 0) return null;
129         Node temp = this.head;
130         this.head = this.head.next;
131         temp.next = null;
132         length--;
133         if (length == 0)
134         {
135             this.tail = null;
```

```
136         }
137         return temp;
138     }
139
140     // RemoveLast(): elimina y regresa el último nodo (null si la lista está vacía).
141     // Se recorre la lista con dos punteros (temp y pre, el anterior) hasta llegar al final;
142     // pre se convierte en la nueva tail y su next se pone en null.
143     // Se corrigió: el nodo removido también se desconecta y, si la lista queda vacía, head y tail van a null.
144     public Node removeLast()  3 usages
145     {
146         if (length == 0) return null;
147         Node temp = this.head;
148         Node pre = this.head;
149         while (temp.next != null)
150         {
151             pre = temp;
152             temp = temp.next;
153         }
154         this.tail = pre;
155         this.tail.next = null;
156         length--;
157         if (length == 0)
158         {
159             this.head = null;
160             this.tail = null;
161         }
162         return temp;
163     }
164
```

```
165     // Remove(indice): elimina y regresa el nodo en la posición "index" (null si el indice es inválido).
166     // - index 0        -> equivale a removeFirst
167     // - index length-1 -> equivale a removeLast
168     // - otro caso      -> el nodo anterior salta al nodo removido, apuntando a su siguiente
169     public Node remove(int index)  2 usages
170     {
171         if (index < 0 || index >= length) return null;
172         if (index == 0) return removeFirst();
173         if (index == length - 1) return removeLast();
174
175         Node pre = getNode( index: index - 1);
176         Node temp = pre.next;
177         pre.next = temp.next;
178         temp.next = null;
179         length--;
180         return temp;
181     }
182
183     // getNode(indice): regresa el nodo en la posición "index" (null si el índice es inválido).
184     // Recorre la lista desde head avanzando "index" veces.
185     public Node getNode(int index)  5 usages
186     {
187         if (index < 0 || index >= length) return null;
188         Node temp = this.head;
189         for (int i = 0; i < index; i++)
190         {
191             temp = temp.next;
192         }
193         return temp;
```

```java

    // setNode(indice, valor): cambia el valor del nodo en la posición "index".
    // Regresa true si lo logró, false si el índice es inválido.

    public boolean setNode(int index, int value)  2 usages
    {
        Node temp = getNode(index);
        if (temp == null) return false;
        temp.value = value;
        return true;
    }

    // Medio(): regresa el nodo de en medio usando dos punteros, sin contadores.
    // "slow" avanza de uno en uno y "fast" de dos en dos. Cuando fast llega al final,
    // slow está en la mitad. Si la cantidad de nodos es par, regresa el de la derecha.

    public Node medio()  4 usages
    {
        if (head == null) return null;
        Node slow = this.head;
        Node fast = this.head;
        while (fast != null && fast.next != null)
        {
            slow = slow.next;
            fast = fast.next.next;
        }
        return slow;
    }
```

```java
    // EsCiclo(): regresa true si la lista tiene un ciclo (algoritmo de la tortuga y la liebre de Floyd).
    // "slow" avanza de uno en uno y "fast" de dos en dos. Si hay ciclo, fast eventualmente
    // alcanza a slow; si no, fast llega a null (fin de la lista).
    public boolean esCiclo()  4 usages
    {
        Node slow = this.head;
        Node fast = this.head;
        while (fast != null && fast.next != null)
        {
            slow = slow.next;
            fast = fast.next.next;
            if (slow == fast) return true;
        }
        return false;
    }

    // Reversa(): invierte el orden de la lista in-place.
    // Primero se intercambian head y tail. Luego se recorre la lista con tres punteros
    // (before, temp, after) y en cada nodo se invierte la dirección de su "next".
    public void reversa()  2 usages
    {
        Node temp = this.head;
        this.head = this.tail;
        this.tail = temp;

        Node after = null;
        Node before = null;
        while (temp != null)
        {
            after = temp.next;  // guardamos el siguiente antes de perderlo
```

```
            after = temp.next;   // guardamos el siguiente antes de perderlo
            temp.next = before;  // invertimos el enlace
            before = temp;       // avanzamos before
            temp = after;        // avanzamos temp
        }
    }

    // printList(): imprime los valores de la lista separados por espacios.
    // Nota: si la lista tiene un ciclo, este método nunca termina.
    public void printList()  6 usages
    {
        Node temp = this.head;
        while (temp != null)
        {
            System.out.print(temp.value + " ");
            temp = temp.next;
        }
        System.out.println();
    }

    // Pruebas rápidas de todos los métodos
    public static void main(String[] args)
    {
        LinkedList list = new LinkedList( value: 2);
        list.append(3);
        list.prepend( value: 1);                        // 1 2 3
        list.insert( index: 3,  value: 5);                    // 1 2 3 5
        list.insert( index: 3,  value: 4);                    // 1 2 3 4 5
```

```
        list.printList();

        System.out.println("Medio (5 nodos): " + list.medio().value);      // 3
        list.append(6);
        System.out.println("Medio (6 nodos): " + list.medio().value);      // 4

        list.setNode( index: 0,  value: 10);                  // 10 2 3 4 5 6
        System.out.println("getNode(0): " + list.getNode( index: 0).value);       // 10

        list.reversa();                       // 6 5 4 3 2 10
        list.printList();

        list.removeFirst();                   // 5 4 3 2 10
        list.removeLast();                    // 5 4 3 2
        list.remove( index: 1);                     // 5 3 2
        list.printList();

        System.out.println("¿Ciclo? " + list.esCiclo());                  // false
        list.getTail().next = list.getHead();  // creamos un ciclo
        System.out.println("¿Ciclo? " + list.esCiclo());                  // true
    }
}
```